第33课 结构体入门

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：个人名片（入门热身）

题目： 定义一个结构体 `Person`，包含姓名（string）和年龄（int）。输入一个人的姓名和年龄，输出 `"xxx is yy years old."`。

讲解思路： 最简单的结构体使用，帮助学生跑通"定义 声明 赋值 访问"的完整流程。

```
#include <bits/stdc++.h>
using namespace std;

struct Person {
 string name;
 int age;
};

int main() {
 Person p;
 cin >> p.name >> p.age;
 cout << p.name << " is " << p.age << " years old." << endl;
 return 0;
}
```

教学话术：

> 这道题很简单对不对？但是它帮你走完了结构体的"四步流程"。记住这个套路，后面的题都是在这个基础上变花样。

例题 2：全班花名册（结构体数组）

题目： 输入 n 个学生的姓名和成绩，输出所有成绩 >= 90 分的学生姓名。

讲解思路： 结构体数组 + 遍历筛选，复习 `if` 判断。

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
 string name;
 int score;
};

int main() {
 int n;
 cin >> n;
 Student stu[105];

 for (int i = 0; i < n; i++) {
 cin >> stu[i].name >> stu[i].score;
 }

 cout << "Honor Roll:" << endl;
 for (int i = 0; i < n; i++) {
 if (stu[i].score >= 90) {
 cout << stu[i].name << endl;
 }
 }

 return 0;
}
```

教学话术：

> 以前我们用两个数组 `name[i]` 和 `score[i]` 也能做。但如果还要加年龄、班级、电话呢？数组越来越多，容易搞混。结构体把它们**打包在一起**，`stu[i]` 就是一个完整的人，清清楚楚。

例题 3：成绩排名（结构体 + sort）

题目： 输入 n 个学生的姓名和成绩，按成绩从高到低排序输出。如果成绩相同，按姓名字典序从小到大排。

讲解思路： 这是结构体最经典的应用场景——CSP-J 中大量排序题都需要这个技巧。

```
#include <bits/stdc++.h>
using namespace std;

struct Student {
 string name;
 int score;
};

bool cmp(Student a, Student b) {
 if (a.score != b.score) return a.score > b.score; // 成绩高的在前
 return a.name < b.name; // 成绩相同，姓名字典序小的在前
}

int main() {
 int n;
 cin >> n;
 Student stu[105];

 for (int i = 0; i < n; i++) {
 cin >> stu[i].name >> stu[i].score;
 }

 sort(stu, stu + n, cmp);

 for (int i = 0; i < n; i++) {
 cout << stu[i].name << " " << stu[i].score << endl;
 }

 return 0;
}
```

教学话术：

> `cmp` 函数就像你当裁判：两个同学站在你面前，你决定谁排前面。先比成绩，成绩不同就高分在前；成绩一样再比名字。`sort` 每次要比较两个人时，就来问你这个"裁判函数"。

附：例题源代码与运行结果

例题 1：Personal Card (Warm-up)

```
// Define a struct Person with name and age. Read input and print formatted
output.
#include <bits/stdc++.h>
using namespace std;

struct Person {
 string name;
 int age;
};

int main() {
 Person p;
 cin >> p.name >> p.age;
 cout << p.name << " is " << p.age << " years old." << endl;
 return 0;
}
```

Sample Input

```
Alice 12
```

Expected Output

```
Alice is 12 years old.
```

例题 2：Class Roster — Filter High Scores

```
// Read n students (name and score), output names of those with score >= 90.

/*
#include <bits/stdc++.h>
using namespace std;

struct Student {
 string name;
 int score;
};

int main() {
 int n;
 cin >> n;
 Student stu[105];

 for (int i = 0; i < n; i++) {
 cin >> stu[i].name >> stu[i].score;
 }

 cout << "Honor Roll:" << endl;
 for (int i = 0; i < n; i++) {
 if (stu[i].score >= 90) {
 cout << stu[i].name << endl;
 }
 }

 return 0;
}
*/
```

Sample Input

```
4
Alice 88
Bob 95
Carol 72
David 91
```

Expected Output

```
Honor Roll:
Bob
David
```

例题 3：Score Ranking (Struct + sort)

```
// Read n students (name and score). Sort by score descending;
// if scores are equal, sort by name in ascending lexicographic order.

/*
#include <bits/stdc++.h>
using namespace std;

struct Student {
 string name;
 int score;
};

bool cmp(Student a, Student b) {
 if (a.score != b.score) return a.score > b.score; // Higher score first
 return a.name < b.name; // Same score: smaller name first
}

int main() {
 int n;
 cin >> n;
 Student stu[105];

 for (int i = 0; i < n; i++) {
 cin >> stu[i].name >> stu[i].score;
 }

 sort(stu, stu + n, cmp);

 for (int i = 0; i < n; i++) {
 cout << stu[i].name << " " << stu[i].score << endl;
 }

 return 0;
}
*/
```

Sample Input

```
4
Alice 88
Bob 95
Carol 88
David 72
```

Expected Output

```
Bob 95
Alice 88
Carol 88
David 72
```